

CS Study

Database - key + 데이터 무결성

목차

1

데이터 무결성이란

2

데이터베이스 key

3

key 관련 고려사항

4

데이터 무결성 관련 고려 사항

목차

1

데이터 무결성이란

2

데이터베이스 key

3

key 관련 고려사항

4

데이터 무결성 관련 고려 사항

데이터 무결성 개념

데이터 무결성 개념

데이터의 **정확성, 일관성, 유효성, 신뢰성 보장**하는 성질

데이터 **처리 과정에서 정확하고 일관성 있게 유지**되도록 보장하는 것

데이터 무결성 중요성

데이터 신뢰성 보장

무결성 보장되지 않으면 **데이터 신뢰 X**
조직 내외부의 **신뢰 구축하는데 필수적**

정확한 의사결정 지원

데이터 오류 있다면 **잘못된 예측**하게 됨
신뢰할 수 있는 데이터로 **잘못된 의사결정 방지**

중복, 오류 최소화

중복, 오류 데이터 많으면 **관리 비용 증가**
저장 공간 절약, 성능 향상

보안 강화

악의적 데이터 변경 막는 방어선 역할
문제 되는 데이터 처리 방지 가능

데이터 무결성 종류

개체 무결성

테이블의 **모든 행이 고유하게 식별**되어야 한다는 규칙
Primary key를 통해 이 규칙을 보장

- Primary key는 **NULL** 값을 가질 수 없음
- Primary key는 테이블 내에서 **항상 고유한 값** 가져야 함
(중복 값 X)



데이터 무결성 종류

참조 무결성

두 테이블 간의 **관계가 항상 일관성을 유지**해야 한다는 규칙
Foreign key를 통해 이 규칙을 보장

- Foreign key 값은 참조하는 테이블의 기본 키 값 중 하나
- 그렇지 않으면 **NULL**이어야 함

데이터 무결성 종류

도메인 무결성

속성 값이 **미리 정해진 도메인 만족**해야 한다는 규칙
도메인 = **값의 범위, 데이터 타입, 허용 값, 형식 등**

- 도메인을 지정해 **도메인에 맞는 올바른 형식의 데이터만 입력되게** 제한



데이터 무결성 종류

사용자 정의 무결성

특정 비즈니스 규칙, 조직의 정책 따라 **사용자가 직접 정의하는 규칙**
개발자가 애플리케이션, DBMS에서 **업무 규칙으로 정의한 무결성**

- **비즈니스 로직에 맞는 복잡한 조건을 설정**
- 개체/참조/도메인 무결성으로 **표현되지 않는 제약 조건을 포함**

목차

1

데이터 무결성이란

2

데이터베이스 key

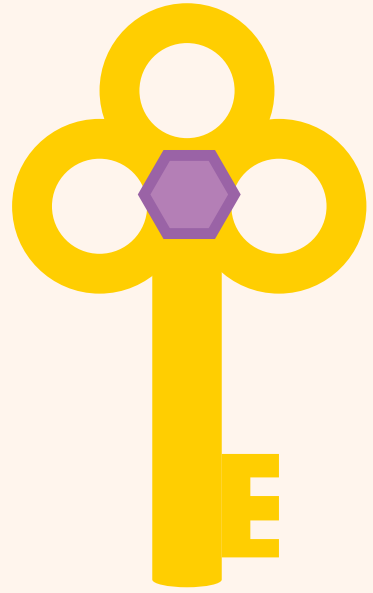
3

key 관련 고려사항

4

데이터 무결성 관련 고려 사항

데이터베이스 key 개념



데이터베이스 key 개념

데이터베이스에서 행을 식별하기 위해

다른 행과 구별 가능한 유일한 기준이 되는 특정 속성을 지정한 것

데이터베이스 key 특징

유일성

키 값은 **테이블 내의 각 레코드 유일하게 구분**할 수 있어야 함

최소성

키는 레코드 **식별하는데 필요한 최소한의 필드로 구성**

불변성

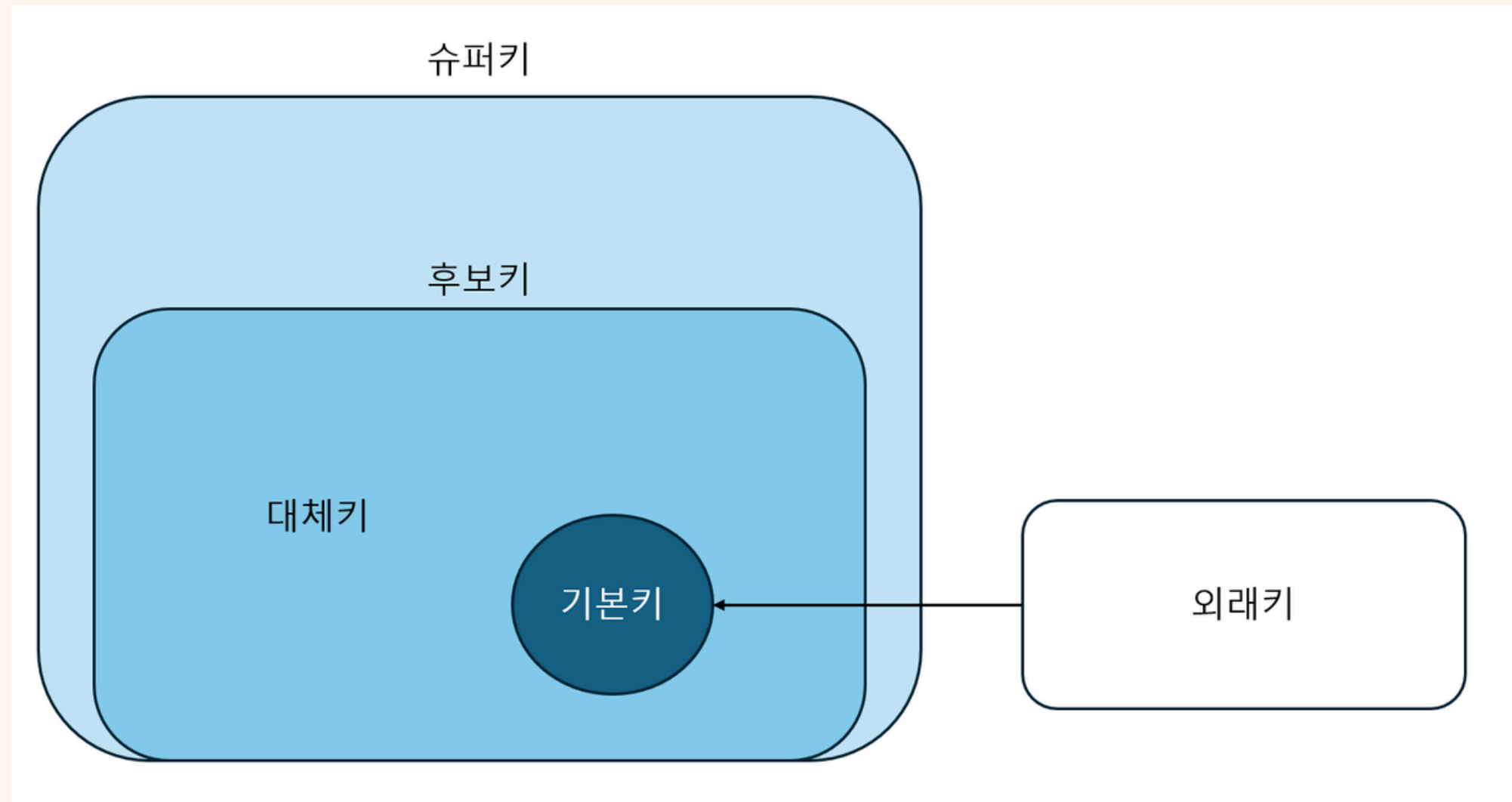
키 값은 **자주 변경되지 않아야 함**



데이터베이스 key 종류

종류

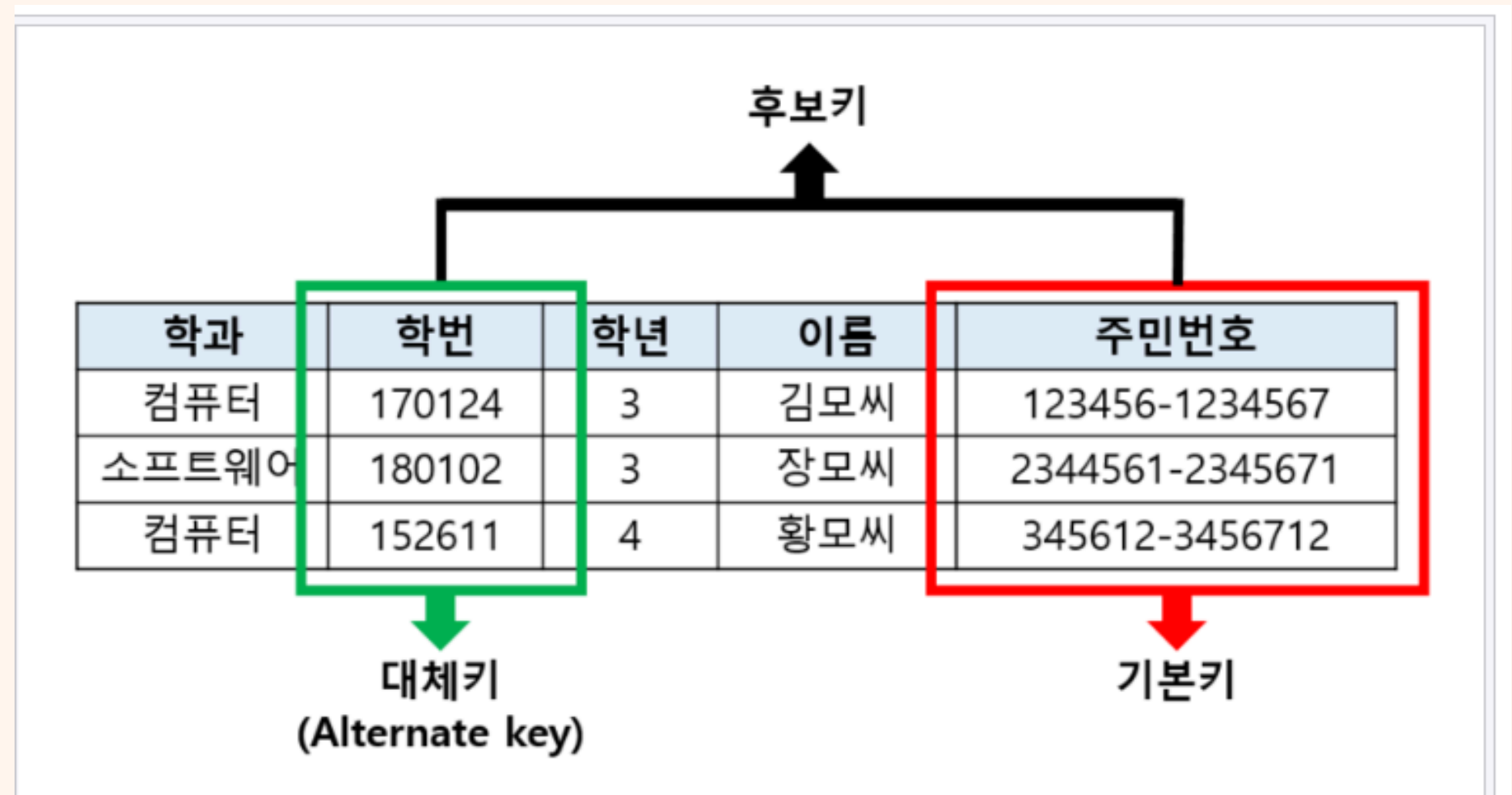
- Super key
- Candidate key
- Primary key
- Alternate key
- Foreign key
- Unique key
- Composite key



데이터베이스 key 종류

Primary key

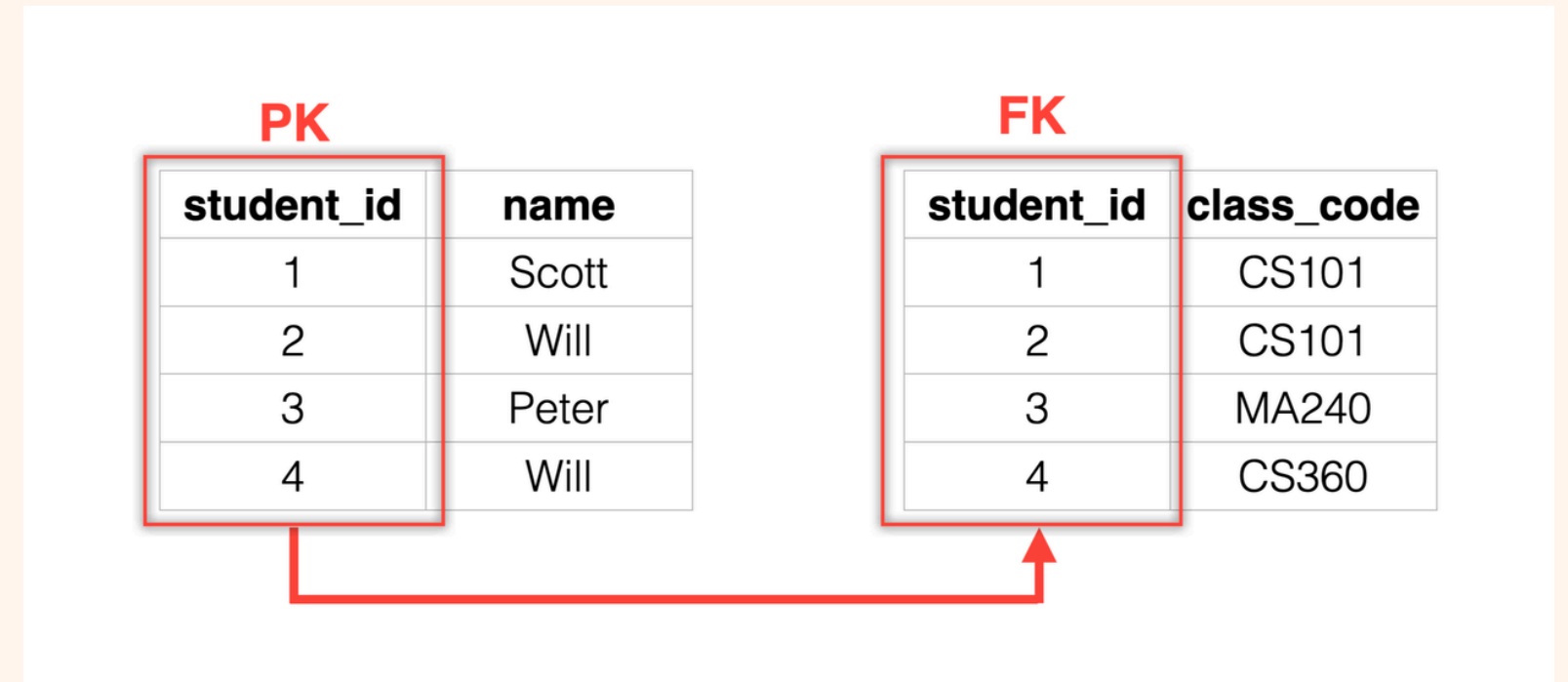
- 테이블의 **모든 데이터를 대표하는 단 하나의** 고유 식별자
- **유일성, 최소성** 만족
- **Not Null**
- **검색, 수정, 삭제의 기준**
- **다른 테이블에서 참조 시 사용**



데이터베이스 key 종류

Foreign key

- 테이블 간의 관계를 맺어주는 키
- 참조하는 부모 테이블 기본키에 존재하는 값
- or NULL 값만 가질 수 있음
- 중복된 값을 가질 수 있음
- 테이블 간 관계 설정
- 두 테이블 연결 (JOIN)



목차

1

데이터 무결성이란

2

데이터베이스 key

3

key 관련 고려사항

4

데이터 무결성 관련 고려 사항

자연키 vs 대리키

자연키

개념

- 실제 데이터 속성 자체가 **의미 가져
고유 식별자로 사용 가능**

예시

- 주민등록번호, 학번, ISBN



자연키 vs 대리키

자연키

장점

- 직관적
- 별도의 공간 불필요

단점

- 데이터 노출
- 변경 가능성 ○
- 성능 비효율
- 고유성 보장 X

자연키 vs 대리키

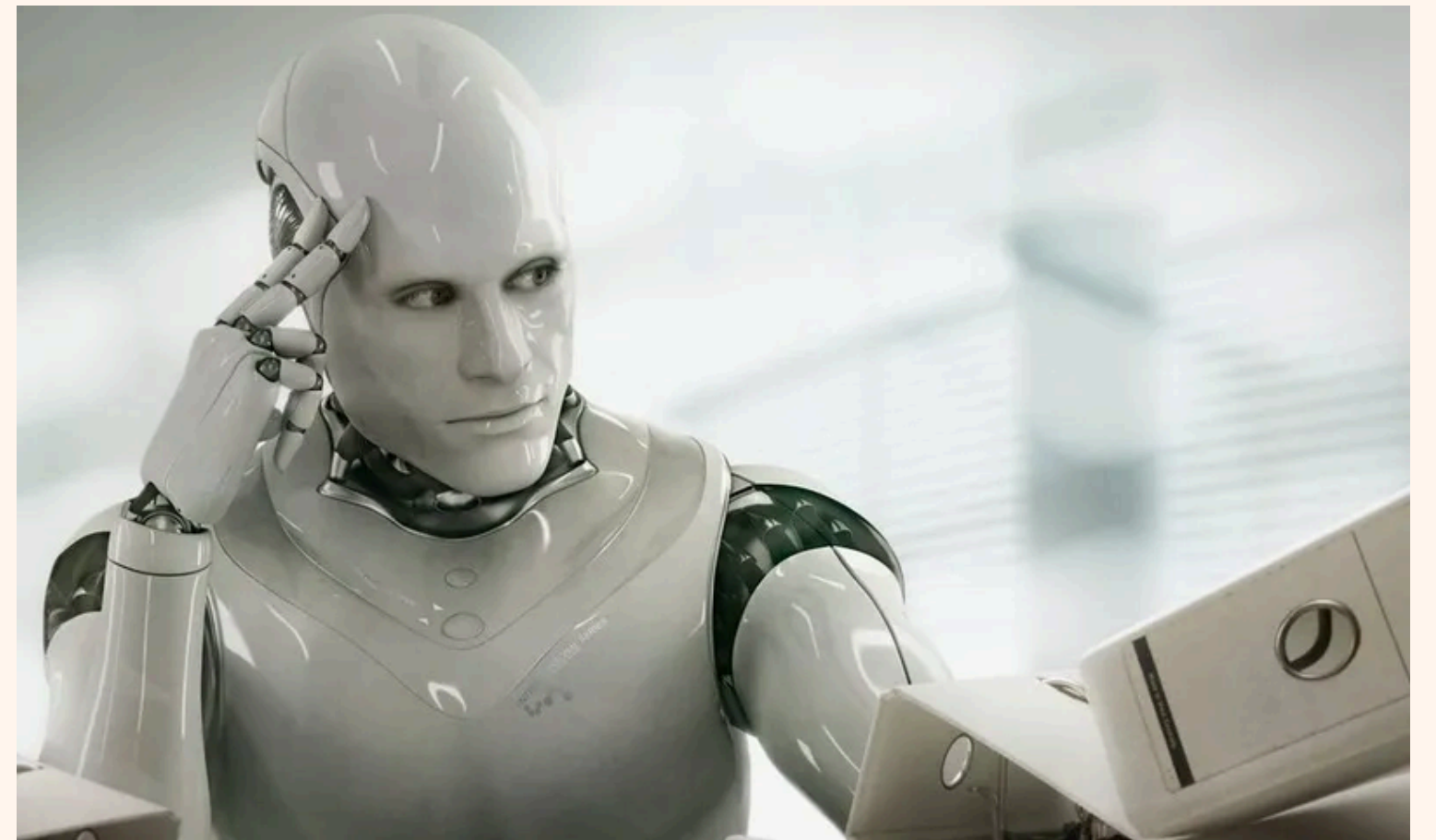
대리키

개념

- 실제 업무 데이터와는 무관한 **DB 시스템** 인위적으로 만들어주는 값을 PK로 사용

예시

- Auto Increment 숫자, UUID



자연키 vs 대리키

대리키

장점

- 불변성
- 유연성
- 성능 개선
- 고유성 보장

단점

- 무의미성
- 추가 공간 필요
- unique 제약 별도 구현

자연키 vs 대리키

PK로 어떤 것 사용?

대부분의 현대 데이터베이스 설계에서는 **대리키 사용을 강력히 권장**

why?

- 비즈니스/업무 규칙, 현실 세계 데이터는 **언제든 변할 수 있음**
- **성능, 보안, 유지보수 관점**에서도 유리
- 자연키가 필요 시 PK로 지정하는 대신 **UNIQUE 제약조건 걸어 대체키로 활용**

FK 사용 시 주의할 점

부모 테이블의 삭제 수정

- 참조 중인 값이 **부모 테이블 (PK)**에서 삭제, 변경 시
- **자식 테이블 (FK)**의 데이터 어떻게 처리할지 결정하는 제약 옵션 반드시 지정해야함
- **ON UPDATE / ON DELETE 옵션**이라고 하며 FK 생성 시 고려
- 종류
 - CASCADE, SET NULL, RESTRICT / NO ACTION

FK 사용 시 주의할 점

부모 테이블의 삭제 수정

- **CASCADE**

- 부모에서 삭제/수정 시, **자식 데이터도 자동 삭제/수정**
- **위험하지만 편리함**
- 의도치 않은 **연쇄 삭제로 대량 데이터 삭제**로 이어질 수 있음
- **명확한 '부모-자식' 관계**(게시글과 댓글 등)에서만 **신중하게 사용**

FK 사용 시 주의할 점

부모 테이블의 삭제 수정

- SET NULL

- 부모 데이터 삭제/수정 시, **자식의 FK 값을 NULL로 변경**
- 단, **FK 컬럼이 NULL 허용**이어야 함

FK 사용 시 주의할 점

부모 테이블의 삭제 수정

- **RESTRICT / NO ACTION**

- 부모 레코드 삭제/수정 시, **자식에 연결된 값이 하나라도 있으면**
- **부모 데이터에 대한 작업 거부**
- **기본 옵션**
- **가장 안전함**
- **무결성 위배 방지**

FK 사용 시 주의할 점

성능 관련

- FK 제약조건 설정 시 DB **INSERT, UPDATE, DELETE** 시 데이터 일관성 항상 검사
- **검사 과정에서 overhead** 발생 가능
- 대규모 데이터에 FK 사용하면 참조 무결성 검사로 성능 저하

FK 사용 시 주의할 점

순환 참조

- 테이블 A가 테이블 B를 참조하고, 동시에 테이블 B가 테이블 A를 참조하는 구조
- 이런 구조는 데이터의 입력, 수정, 삭제를 매우 복잡하고 어렵게 만듦
- 테이블 간 FK로 인한 순환 참조 구조는 설계에서 가급적 피해야 함

Index와 Key

PK에서 Index

- 기본키, 유니크 키는 선언 시 **DB가 자동으로 clustered index 생성**
 - 데이터 들어올 때마다 **테이블 전체 뒤져 중복 값 확인하는 것은 매우 비효율적**
 - **미리 정렬된 인덱스를 사용**하면 중복 여부를 거의 즉시 확인할 수 있음
- **복합 PK를 만들면 그 속성 조합으로 자동으로 복합 인덱스 생성**

Index와 Key

FK에서 Index

- FK 컬럼에도 **인덱스 생성이 권장**
 - FK는 **자동으로 인덱스가 생성되지 않음**
- **why?**
 - FK는 **JOIN** 이나 **WHERE 절**에서 매우 빈번하게 사용
 - 부모 테이블의 데이터 수정/삭제 시, **자식 테이블에 참조하는 데이터 있는지 확인**

Index와 Key

FK에서 Index

- FK는 JOIN 이나 WHERE 절에서 매우 빈번하게 사용
 - 테이블 관계를 확인할 때마다 **Full Table Scan**
 - FK 열에 인덱스가 없으면, **성능이 급격히 저하**
- 부모 테이블의 데이터 수정/삭제 시, 자식 테이블에 참조하는 데이터 있는지 확인
 - FK 열에 인덱스가 없으면 **락 때문에 성능이 저하**
 - 자식 테이블 **full table scan**해야 해서 불필요한 큰 **Lock**을 걸어야 함

복합키 장단점

복합키 장점

- 논리적이고 명확한 데이터 구조
 - **비즈니스 관계**를 데이터 모델에 가장 **자연스럽게 표현 가능**
- 데이터 무결성 보장
 - **단일 컬럼으로는 유일성을 보장할 수 없을 때** 해결책
- 추가 컬럼 불필요
 - **대리키를 따로 만들 필요가 없음**
- 인덱스 효율성
 - **복합 인덱스가 자동으로 생성**

복합키 장단점

복합키 단점

- 복잡한 테이블 관계와 구조
 - 참조하는 테이블에도 **복합키를 구성하는 모든 열을 포함**해야 함
- 성능 저하 가능성
 - **여러 개의 열을 조합한 키**로 JOIN 해야 함
 - **컬럼 많고 컬럼 값 길어질수록** 성능 저하
- ORM 복잡성 증가
 - **훨씬 복잡한 매핑** 과정 필요

목차

1

데이터 무결성이란

2

데이터베이스 key

3

key 관련 고려사항

4

데이터 무결성 관련 고려 사항

무결성 보장 방법

데이터베이스 레벨에서 보장

- DBMS가 **제공하는 기능**을 이용
- 데이터의 규칙 **강제하는 가장 강력하고 안정적인 방법**
- **방법**
 - 제약 조건 설정
 - 데이터 타입, default 값 지정
 - 트리거 사용
 - 트랜잭션 관리

무결성 보장 방법

데이터베이스 레벨에서 보장

- 제약조건 설정 방법
 - 테이블의 **열에 특정 규칙**을 직접 부여
 - 잘못된 데이터 입력 원천 차단
 - **기본 키, 외래 키, 고유 키, Not Null 등**

무결성 보장 방법

애플리케이션 레벨에서 보장

- 데이터가 **데이터베이스에 도달하기 전** 유효성 검증 방법
- **웹/앱 서버 등에서** 데이터의 유효성을 검증하는 방법
- 방법
 - 비즈니스 로직에서 유효성 검사
 - 프레임워크 기능 사용
 - 서버 단에서 유효성 검사
 - 프론트엔드 레벨에서 유효성 검사

무결성 보장 방법

애플리케이션 레벨에서 보장

- 프레임워크 기능 사용
 - JPA의 @NotNull, @Size 등
- 서버 단에서 유효성 검사
 - spring validation, DTO 검증 등

트랜잭션 성능에 무결성이 미치는 영향

무결성이 미치는 영향

- 트랜잭션 성능에 '데이터 신뢰도를 위한 성능 저하'란 직접적인 영향을 미침
- 정확성과 일관성을 강하게 보장할수록 트랜잭션 처리 속도는 느려짐
- 모든 데이터 변경 작업마다 무결성 규칙 깨지지 않는지 검사하는 추가 작업을 수행하기 때문

트랜잭션 성능에 무결성이 미치는 영향

무결성이 미치는 영향

- 성능에 영향 주는 원인들
 - 제약 조건 검사
 - 인덱스 유지 비용
 - 락 관리 증가
 - 트리거 실행

트랜잭션 성능에 무결성이 미치는 영향

무결성이 미치는 영향

- 제약 조건 검사
 - PK, FK, UNIQUE 같은 무결성 제약 조건은
INSERT/UPDATE/DELETE 시마다 검사 필요
- 인덱스 유지 비용
 - 트랜잭션에서 데이터 추가, 수정, 삭제 시
 - **실제 데이터 변경 + 인덱스 갱신, 정렬 동시 발생**

트랜잭션 성능에 무결성이 미치는 영향

무결성이 미치는 영향

- 락 관리 증가

- 부모 삭제 시 → **자식 레코드 참조 검사 필요** →
- FK에 인덱스 없으면 **테이블 스캔** → **락 오래 유지**

- 트리거 실행

- **복잡한 무결성 규칙을 위해** 트리거를 사용하는 경우
- **로직 복잡할수록** 트랜잭션 성능 저하

Quiz

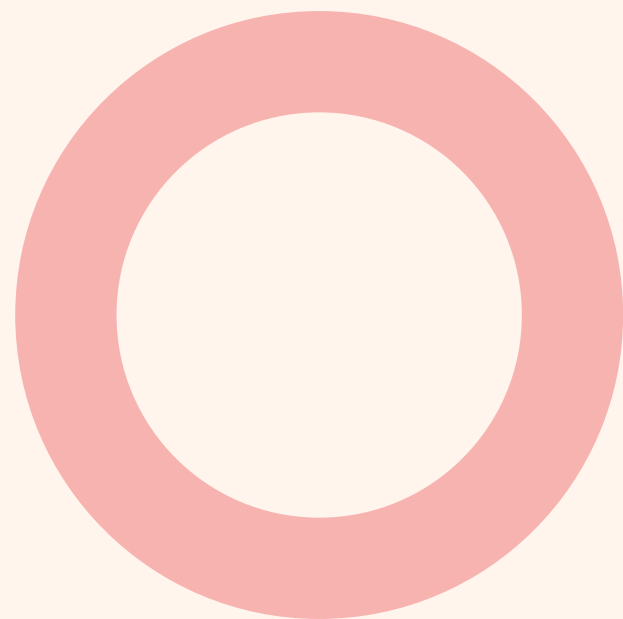
1번 문제

자연키와 대리키 중 자연키를 사용하는 것이 요즘 트렌드



1번 문제

비즈니스/업무 규칙, 현실 세계 데이터는 언제든지 변할 수 있음
자연키는 이러한 변화에 매우 취약
대리키는 비즈니스와 완전히 분리되어 있어 영향 X



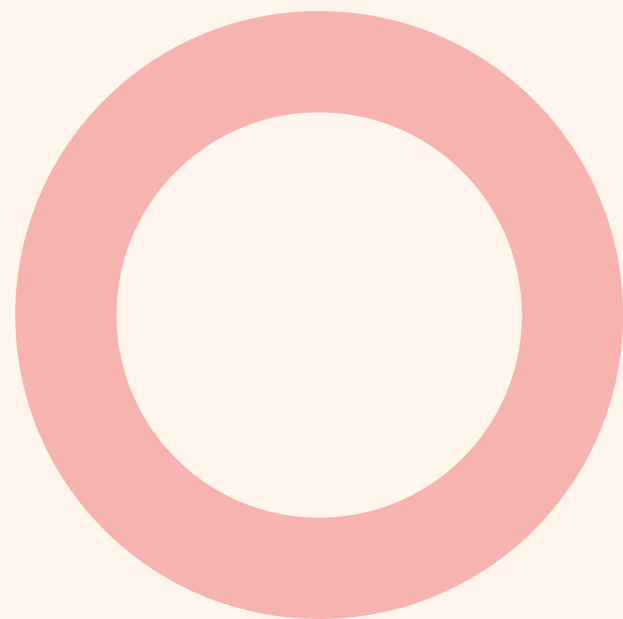
2번 문제

FK에는 인덱스가 있어도 되고 없어도 된다.



2번 문제

FK는 **JOIN의 WHERE 절**에서 사용해 없다면 **Full Table Scan**
부모 테이블의 **데이터 수정/삭제 시**, 자식 테이블에 **참조하는 데이터**
있는지 확인해 **큰 범위에 Lock** 걸어야 함



3번 문제

트랜잭션 성능에 데이터 무결성이 영향을 미친다



3번 문제

데이터의 정확성과 일관성을 **강하게 보장할수록**

트랜잭션 처리 속도는 느려짐

제약조건 검사, 인덱스 유지 비용, lock 관리 비용 등

